claude-windows / 6a82308f-69d6-43a8-9ea4-d8eaf497253d

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6a82308f-69d6-43a8-9ea4-d8eaf497253d\subagents\workflows\wf_3d536535-241\agent-a9ab8cd634ea5a2ef.jsonl`
- SHA-256: `4756ffacd0e31e74cd7c439ed3f6efdf557845625926e82af92b2179444ce287`
- Source modified: `2026-06-16T03:23:04+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_3d536535-241`
- session_id: `6a82308f-69d6-43a8-9ea4-d8eaf497253d`

Transcript

1. user (2026-06-16T03:21:40.636Z)

Adversarial Claim Verifier (voter 1/3)

Be SKEPTICAL. Try to REFUTE this claim. ?2/3 refutations kill it.

Research question

Research question: **What concrete, recent (2022?2025) techniques and evaluation practices should a local, commercially-licensed badminton rally-detector adopt to close the quality gap with a strong cloud VLM (Gemini), given a tiny labeled set (~6 ? ~16 golden videos)?** Produce an adversarially-verified, cited report organized around the four dimensions below. Prefer specific named methods, papers (arXiv/venue), and reported numbers over generic advice; flag anything you cannot verify.

CONTEXT (so you target the real gap, not generic ML):
- System: a local pipeline detects the shuttle per-frame (WASB / TrackNet, MIT-licensed) ? a heuristic turns the trajectory into candidate rally windows ? today an optional Gemini "handoff" confirms rallies. We just measured a fully-local (no-AI) run vs Gemini on 3 matches: local emits far FEWER but MUCH LONGER windows (mean 65s vs 6.1s; one window spanned a whole 174s clip) ? it does NOT miss activity, it FAILS TO SEGMENT. Root cause (code-confirmed): a frame is "active" iff the shuttle is visible AND moving above a small per-second-normalized velocity threshold; active frames within a 2 s gap are merged; there is NO max-window cap and NO inactivity/boundary split. So "shuttle is moving" is conflated with "rally in progress," and dead-time + serves + knock-ups bridge rallies into mega-windows. Shuttle tracking itself is excellent (96?99% per-frame visibility).
- HARD CONSTRAINTS: weights/data/code must be MIT/Apache-2.0/BSD only (no viral/NC/custom licenses, reject Gemma-3/Llama-4-MAU-cap); we may NEVER train owned weights on paid-LLM (Gemini/OpenAI/Anthropic) outputs (terms/copyright), though paid LLMs may be used at inference; minors excluded from any training pool; runs on a single local GPU.
- WHAT WE ALREADY HAVE (do NOT re-survey the basics ? go deeper/newer): the architecture skeleton is known to be standard Temporal Action Localization/Segmentation (proposal+scorer) + late/score-level fusion + stacked generalization; in-domain prior art (MonoTrack, ShuttleSet, See et al. 2024/25 non-broadcast badminton rally start/end, Ghosh et al. point-segmentation, tennis/TT play-break state machines) is already cited. We have a working eval harness: leave-one-video-out (grouped by match), temporal-IoU F1 @0.5, F1@{0.1,0.25,0.5}, mAP@tIoU, segment-count-ratio, bootstrap 95% CIs, paired exact sign-test, Platt/isotonic calibration + Brier/ECE, a tiny numpy LogisticRegression scorer over per-window feature columns, an experiment leaderboard, and an "eval?serve" guard (a cue measured +0.074 F1 on permissive proposal windowing but ?0.008 on the coarser served windowing ? reverted). Available but default-OFF per-window signals: net-crossing count (rally gate), 5 person/court features, 3 audio-hit features.

THE FOUR DIMENSIONS:

1) WINDOWING & BOUNDARIES (Temporal Action Localization/Segmentation): beyond proposal+score ? what are the best *recent* methods for (a) turning a dense per-frame signal into well-segmented

actions, (b) explicitly controlling OVER-segmentation/merging, (c) boundary detection/regression and splitting merged segments? Cover e.g. MS-TCN/MS-TCN++, ASRF (action-boundary regression), BMN/BSN boundary-matching, ActionFormer/TriDet/TemporalMaxer (actionness + boundary heads), smoothing/boundary losses, and the standard metrics for over-segmentation (segmental F1@k, edit/Levenshtein score, mAP@tIoU). Which transfer to a 1-D shuttle-trajectory + auxiliary-cue setting on tiny data?

2) SHUTTLE-TRAJECTORY ? RALLY SIGNAL PROCESSING (racket-sports specific): how do published systems convert ball/shuttle tracking into rally/point segments and distinguish "in-play" from dead-time? Cover rally-state machines, serve/let detection, net-crossing & two-sided-occupancy cues, hit/contact (audio + kinematic) detection, and fps-invariant trajectory features. What features best separate "rally" from "shuttle merely moving"?

3) SMALL-DATA STRATEGIES: with ~6?16 labeled videos, what gives the most quality per label ? active learning (which videos/segments to label next; uncertainty/diversity/core-set), semi-/weakly-/point-supervised TAL (e.g. SF-Net, timestamp supervision), self-training/pseudo-labeling, and data-efficient TAL? Separately: the legally-clean ways to exploit a strong teacher (Gemini) at INFERENCE for weak/pseudo-labels or as an evaluation oracle WITHOUT its outputs becoming training data for owned weights ? what does the literature say about teacher-student / distillation boundaries and label-noise handling, and what governance distinctions matter?

4) EVALUATION METHODOLOGY & RIGOR at small n: trustworthy measurement with ~6?16 videos ? bootstrap/permutation tests, paired sign-tests, leave-one-group-out vs RLOOCV bias, calibration (reliability, ECE), the over-segmentation-blindness of plain tIoU, shadow/A-B evaluation, regression gates, and concrete pitfalls (label noise, annotator agreement/IAA, multiple-comparison/over-fitting traps, eval-vs-serve drift). What metrics + protocols should a small-data sports-segmentation leaderboard standardize on?

Deliverable: a structured, cited report with, per dimension, the 3?5 highest-leverage techniques that APPLY to our constrained setting (small data, local, license-clean, 1-D trajectory + cues + learned scorer), each with: what it is, why it fits our over-merge/small-data problem, the expected difficulty/payoff, and the citation. Call out which ideas are low-effort high-confidence wins vs research bets. Be explicit about anything unverified or license-incompatible.

Claim under review

"On THUMOS14, TriDet reaches 69.3% average mAP, beating the prior best (ActionFormer at 66.8%) by 2.5 points while using only 74.6% of its latency, showing a convolution-only (no self-attention) detector can outperform transformer-based TAD at lower cost."

Source: https://openaccess.thecvf.com/content/CVPR2023/papers/Shi_TriDet_Temporal_Action_Detection_With_Relative_Boundary_Modeling_CVPR_2023_paper.pdf (primary)
Supporting quote: "TriDet hits an average mAP of 69.3% on THUMOS14, outperforming the previous best by 2.5%, but with only 74.6% of its latency."

Checklist

1. Is the claim actually supported by the quote, or is it an overreach/misread?
2. WebSearch for contradicting evidence ? does any credible source dispute or heavily qualify this?
3. Is the source quality sufficient for the claim's strength? (extraordinary claims need primary sources)
4. Is the claim outdated? (check dates ? old claims about fast-moving fields are suspect)
5. Is this a marketing claim / press release / cherry-picked benchmark / forum speculation?

refuted=true if: unsupported by quote / contradicted / low-quality source for strong claim / outdated / marketing fluff.
refuted=false ONLY if: claim is well-supported, current, and source quality matches claim strength.
Default to refuted=true if uncertain.

Structured output only. Evidence MUST be specific.

2. assistant (2026-06-16T03:21:43.979Z)

I'll verify this claim about TriDet's performance on THUMOS14 by checking the primary source.

3. assistant (2026-06-16T03:21:46.943Z)

I'll fetch the primary source (the TriDet CVPR 2023 paper) to verify the specific numbers in the claim.

4. user (2026-06-16T03:21:56.111Z)

The server returned HTTP 403 Forbidden.

The response body was not retrieved. If this URL requires authentication, use an authenticated tool (e.g. `gh` for GitHub, or an MCP-provided fetch tool) instead of WebFetch.

5. assistant (2026-06-16T03:22:02.779Z)

The CVF page blocked direct fetch. Let me try the arXiv version and search for the numbers.

6. user (2026-06-16T03:22:06.900Z)

TriDet Performance Metrics on THUMOS14

Based on the abstract, here are the exact reported numbers:

(1) Average mAP value:
"TriDet hits an average mAP of $69.3\\%$ on THUMOS14"

(2) Comparison baseline:
The abstract does not explicitly name which prior method TriDet is compared against, only stating it outperforms "the previous best."

(3) Margin of improvement:
"outperforming the previous best by $2.5\\%$"

(4) Latency comparison:
"with only $74.6\\%$ of its latency" (referring to the computational cost relative to the previous best method)

The abstract emphasizes that TriDet achieves superior performance while being significantly more efficient, though it does not identify the specific competing method by name in this summary.

7. user (2026-06-16T03:22:15.792Z)

Web search results for query: "TriDet Temporal Action Detection Relative Boundary Modeling THUMOS14 average mAP 69.3 ActionFormer latency"

Links: [{"title":"TriDet: Temporal Action Detection with Relative Boundary Modeling | OpenReview","url":"https://openreview.net/forum?id=3C1Gs6vop9"},{"title":"TriDet: Temporal Action Detection with Relative Boundary Modeling","url":"https://arxiv.org/pdf/2303.07347"},{"title":"TriDet: Temporal Action Detection","url":"https://www.emergentmind.com/papers/2303.07347"},{"title":"[2303.07347] TriDet: Temporal Action Detection with Relative Boundary Modeling","url":"https://arxiv.org/abs/2303.07347"},{"title":"TriDet: Temporal Action Detection with Relative Boundary Modeling Dingfeng Shi?","url":"https://openaccess.thecvf.com/content/CVPR2023/papers/Shi_TriDet_Temporal_Action_Detection_With_Relative_Boundary_Modeling_CVPR_2023_paper.pdf"},{"title":"TriDet: Temporal Action Detection with Relative Boundary Modeling - ADS","url":"https://ui.adsabs.harvard.edu/abs/2023arXiv230307347S/abstract"},{"title":"Paper page - TriDet: Temporal Action Detection with Relative Boundary Modeling","url":"https://paperswithcode.com/paper/tridet-temporal-action-detection-with"},{"title":"CVPR Poster TriDet: Temporal Action Detection With Relative Boundary Modeling","url":"https://cvpr.thecvf.com/virtual/2023/poster/21956"},{"title":"GitHub - dingfengshi/TriDet: [CVPR2023] Code for the paper, TriDet: Temporal Action Detection with Relative Boundary Modeling · GitHub","url":"https://github.com/dingfengshi/TriDet"},{"title":"Benchmarking Data Efficiency and Computational Efficiency of Temporal Action Localization

Models","url":"https://arxiv.org/pdf/2308.13082"}]

Based on the search results, I found comprehensive information about TriDet and its comparison with ActionFormer:

Key Findings:

TriDet achieves an average mAP of 69.3% on THUMOS14, outperforming the previous best by 2.5%, but with only 74.6% of its latency.

The "previous best" method referenced is ActionFormer. According to the paper's comparison table, ActionFormer achieved 66.8% average mAP on THUMOS14 with a latency of 224ms, while TriDet achieved 69.3% mAP with a latency of 167ms.

Method Overview:

TriDet is a one-stage framework for temporal action detection that proposes a novel Trident-head to model the action boundary via an estimated relative probability distribution around the boundary. The feature pyramid of TriDet includes an efficient Scalable-Granularity Perception (SGP) layer to mitigate the rank loss problem of self-attention and aggregate information across different temporal granularities.

This represents a significant improvement in both accuracy and computational efficiency compared to ActionFormer on the THUMOS14 benchmark.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

8. assistant (2026-06-16T03:22:19.052Z)

The abstract is confirmed. Now let me verify the key sub-claim: that the "previous best" being compared against is specifically ActionFormer at 66.8% mAP. The abstract doesn't name it, so I need to confirm the latency comparison and the ActionFormer baseline number from the results tables.

9. user (2026-06-16T03:22:25.538Z)

I appreciate your question, but I must be transparent: the provided content is a PDF file encoded in binary/compressed format that I cannot reliably parse into readable text. The document appears to contain research paper content about video action detection methods, but I cannot extract specific numerical values from tables with confidence.

To answer your questions accurately about:
1. ActionFormer's mAP on THUMOS14
2. TriDet's mAP on THUMOS14
3. Specific latency comparisons and the 74.6% reference

I would need access to a readable text version of the paper or a clearer image of the relevant tables. I cannot responsibly quote specific numbers from data I cannot verify visually.

I recommend providing either:
- A text-based version of the paper
- Screenshots of the specific tables
- A link to the published paper

This ensures accuracy in citing benchmark results.

[Binary content (application/pdf, 1.4MB) also saved to C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6a82308f-69d6-43a8-9ea4-d8eaf497253d\tool-results\webfetch-1781579910495-xf7r2b.pdf]

10. assistant (2026-06-16T03:22:28.254Z)

The PDF was downloaded. Let me read it directly to verify the table numbers.

11. user (2026-06-16T03:22:29.041Z)

pdftoppm failed: Command 'pdftoppm' not found or is in an unsafe location (current directory)

12. assistant (2026-06-16T03:22:31.469Z)

The PDF reader needs poppler. Let me check if it's available, otherwise extract text another way.

13. user (2026-06-16T03:22:32.975Z)

pdfplumber available
PyPDF2 available
fitz available

14. user (2026-06-16T03:22:37.841Z)

'69.3% on THUMOS14, outperforming the previous best by'
'2.5%, but with only 74.6% of its latency. The code is re-'
'like layers (e.g. ActionFormer [49]).'
'AvgPool'
'\\p h i (x) &= ReLU(FC(AvgPool(x))),\\\\ \\psi (x) &= Conv_w(x),'
'where AvgPool(x) is the average pooling for all features'
'Avg.'
'ActionFormer [49]'
'66.8'
'69.3'
'0.75 0.95 Avg.'
'0.5 Avg.'
'ActionFormer [49] 26.6 25.4 24.2 22.3 19.1 23.5'
'ActionFormer [49] 25.2 24.1 22.7 20.5 17.0 21.9'
'method achieves an average mAP of 69.3%, outperform-'
'0.75 0.95 Avg.'
'ActionFormer [49]'
'The method of our main comparison is ActionFormer [49],'
'ActionFormer with the same features by a large margin'
'Avg.'
'82.1 71.0 43.9 66.8'
'83.6 72.9 47.4 69.3'
'Latency'
'0.7 Avg. Main Head All'
'ActionFormer 82.1 43.9 66.8 30.8'
'83.6 47.4 69.3 14.5'
'tional complexity and latency of TriDet with the recent Ac-'
'for each part and the inference latency (average over five'
'69.3'
'Avg. Bin'
'Avg.'
'69.3'
'69.3'
'the ActionFormer (14.5 versus 30.8), and the overall la-'
'and the GMACs is still 1.6G smaller than ActionFormer,'
'and the latency is still only 74.6% of it, proving the high'
'Avg.'
'0.75 0.95 Avg.'
'69.3'
'Avg.'
'66.8'

15. assistant (2026-06-16T03:22:44.783Z)

All key numbers are now confirmed directly from the primary source PDF. Let me note the latency

table detail to verify the "74.6%" interpretation precisely.

The extracted text confirms:
- TriDet THUMOS14 avg mAP = **69.3** (`83.6 72.9 47.4 69.3`)
- ActionFormer THUMOS14 avg mAP = **66.8** (`82.1 71.0 43.9 66.8`)
- Margin: 69.3 ? 66.8 = **2.5 points** ("outperforming the previous best by 2.5%")
- Latency table: `ActionFormer 82.1 43.9 66.8 30.8` and TriDet `83.6 47.4 69.3 14.5` ? the "Head" latency is 14.5 vs 30.8, but the abstract/text claim "the latency is still only 74.6% of it" refers to the **overall** latency relative to ActionFormer.

Let me verify the exact framing of the 74.6% ? whether it's overall latency vs ActionFormer specifically.

16. user (2026-06-16T03:22:47.079Z)

TriDet, we
propose an efficient Scalable-Granularity Perception (SGP)
layer to mitigate the rank loss problem of self-attention that
takes place in the video features and aggregate information
across different temporal granularities. Benefiting from the
Trident-head and the SGP-based feature pyramid, TriDet
achieves state-of-the-art performance on three challeng-
ing benchmarks: THUMOS14, HACS and EPIC-KITCHEN
100, with lower computational costs, compared to previ-
ous methods. For example, TriDet hits an average mAP of
69.3% on THUMOS14, outperforming the previous best by
2.5%, but with only 74.6% of its latency. The code is re-
leased to https://github.com/dingfengshi/TriDet.
1. Introduction
Temporal action detection (TAD) aims to detect all start
and end instants and corresponding actio

17. assistant (2026-06-16T03:22:55.082Z)

All elements of the claim are verified directly from the primary source (the TriDet CVPR 2023 paper PDF).

Verification summary:
- **69.3% avg mAP on THUMOS14** ? confirmed (`83.6 72.9 47.4 69.3`)
- **ActionFormer at 66.8%** ? confirmed as the explicitly named baseline ("The method of our main comparison is ActionFormer") with `82.1 71.0 43.9 66.8`
- **+2.5 points margin** ? confirmed (69.3 ? 66.8 = 2.5; text: "outperforming the previous best by 2.5%")
- **74.6% of its latency** ? confirmed ("with only 74.6% of its latency", and "the latency is still only 74.6% of it" referring to ActionFormer)
- **Convolution-only / no self-attention** ? confirmed: SGP layer is designed "to mitigate the rank loss problem of self-attention," i.e., it replaces self-attention with a convolutional design

One minor nuance: the abstract's "previous best" is not named in the abstract itself, but the body explicitly identifies ActionFormer as "the method of our main comparison" and the THUMOS14 tables show ActionFormer at 66.8 as the top prior method ? so attributing the comparison to ActionFormer is fully supported. The "74.6% of its latency" is the overall latency ratio vs ActionFormer (the head-only latency is 14.5 vs 30.8). The claim is accurate, current, and from a peer-reviewed primary source (CVPR 2023).

18. user (2026-06-16T03:23:00.653Z)

Structured output provided successfully

19. assistant (2026-06-16T03:23:04.346Z)

Verified the claim against the primary source. All four numeric components (69.3% mAP, 66.8% ActionFormer baseline, +2.5 point margin, 74.6% latency) and the convolution-only/no-self-

attention characterization are accurately supported by the TriDet CVPR 2023 paper. Claim NOT refuted (confidence: high).